

Sistema de Controle Financeiro com Banco de Dados

Sistema completo de controle financeiro pessoal com backend Node.js, banco de dados SQLite e interface web moderna.

Pré-requisitos

Antes de começar, você precisa ter instalado em seu computador:

1. **Node.js** (versão 14 ou superior)

- Download: <https://nodejs.org/>
 - Para verificar se já tem instalado, abra o terminal e digite: `node --version`
-

Instalação Passo a Passo

Passo 1: Criar a estrutura de pastas

Crie uma pasta para o projeto e organize assim:

```
controle-financeiro/  
├── server.js  
├── package.json  
└── public/  
    └── index.html
```

Passo 2: Criar os arquivos

1. Crie o arquivo `package.json` (copie o conteúdo do artefato "package.json")
2. Crie o arquivo `server.js` (copie o conteúdo do artefato "server.js")
3. Crie a pasta `public` dentro da pasta do projeto
4. Crie o arquivo `index.html` dentro da pasta `public` (copie o HTML atualizado)

Passo 3: Instalar dependências

Abra o terminal na pasta do projeto e execute:

```
bash  
  
npm install
```

Isso vai instalar todas as bibliotecas necessárias:

- express (servidor web)
- sqlite3 (banco de dados)
- cors (permitir requisições do frontend)
- body-parser (processar dados JSON)

Passo 4: Iniciar o servidor

No terminal, execute:

```
bash  
npm start
```

Você verá algo como:

```
✓ Conectado ao banco de dados SQLite  
✓ Tabela transactions pronta  
🚀 Servidor rodando em http://localhost:3000  
📊 API disponível em http://localhost:3000/api/transactions
```

Passo 5: Acessar o sistema

Abra seu navegador e acesse:

```
http://localhost:3000
```

🎯 Como Funciona

Arquitetura do Sistema



Endpoints da API

Método	Endpoint	Descrição
GET	/api/transactions	Lista todas as transações
GET	/api/transactions/:id	Busca uma transação específica
POST	/api/transactions	Cria nova transação
PUT	/api/transactions/:id	Atualiza uma transação
DELETE	/api/transactions/:id	Deleta uma transação
DELETE	/api/transactions	Deleta todas as transações
GET	/api/summary	Retorna resumo financeiro

 Estrutura do Banco de Dados

Tabela: transactions

Campo	Tipo	Descrição
id	INTEGER	Chave primária (auto incremento)
data	DATE	Data da transação
descricao	TEXT	Descrição da transação
categoria	TEXT	Categoria (Alimentação, Transporte, Ações, etc)
valor	REAL	Valor em reais
tipo	TEXT	'receita', 'despesa' ou 'investimento'
created_at	DATETIME	Data/hora de criação

Tipos de Transação:

- **Receita:** Dinheiro que entra (salário, freelance, rendimentos)
 -  Aumenta o saldo disponível
- **Despesa:** Dinheiro que sai (contas, compras, alimentação)
 -  Diminui o saldo disponível
- **Investimento:** Dinheiro aplicado (ações, fundos, tesouro direto)
 -  Contabilizado separadamente
 -  Não afeta o saldo disponível (está "guardado")

Fórmula do Saldo Disponível: Receitas - Despesas

Comandos Úteis

Iniciar servidor

```
bash  
npm start
```

Iniciar com auto-reload (desenvolvimento)

```
bash  
npm run dev
```

(Usa nodemon - reinicia automaticamente ao salvar alterações)

Parar o servidor

Pressione `Ctrl + C` no terminal

Backup e Restauração

Fazer Backup

O arquivo do banco de dados fica em: `financeiro.db`

Para fazer backup, simplesmente copie este arquivo para outro local:

```
bash  
cp financeiro.db backup-financeiro-2024-01-05.db
```

Restaurar Backup

```
bash
```

```
cp backup-financeiro-2024-01-05.db financeiro.db
```

Limpar Banco (Recomeçar)

Apenas delete o arquivo `financeiro.db` - ele será recriado automaticamente na próxima inicialização.



Solução de Problemas

Erro: "Cannot find module 'express'"

Solução: Execute `npm install` novamente

Erro: "Port 3000 is already in use"

Solução: Outro programa está usando a porta 3000

- Opção 1: Feche outros servidores Node.js
- Opção 2: Mude a porta no `server.js` (linha `const PORT = 3000;`)

Frontend não conecta ao backend

Verificações:

1. Servidor está rodando? (deve ver "Servidor rodando" no terminal)
2. URL correta? (<http://localhost:3000>)
3. Firewall não está bloqueando?

Dados não aparecem após adicionar

Solução:

1. Abra o Console do navegador (F12)
 2. Veja se há erros
 3. Verifique se o badge mostra "Servidor Online" (verde)
-



Recursos Implementados



Cadastro de receitas, despesas e investimentos



Três tipos de transação:

- **Receitas** (aumentam saldo)
 - **Despesas** (diminuem saldo)
 - **Investimentos** (separados do saldo disponível) ☒ Cálculo automático de saldo disponível
 - ☒ Contabilização separada de investimentos
 - ☒ Categorização de transações (incluindo 8+ categorias de investimentos)
 - ☒ Persistência em banco de dados SQLite
 - ☒ Interface responsiva (funciona em mobile)
 - ☒ Indicador de status do servidor
 - ☒ Validações de dados
 - ☒ Formatação de valores em Reais
 - ☒ Exclusão de transações
 - ☒ Resumo financeiro com 4 métricas principais
-

Segurança

IMPORTANTE:

- Este sistema é para uso **LOCAL** (computador pessoal)
 - **NÃO exponha** diretamente à internet sem segurança adicional
 - Para uso em produção, adicione:
 - Autenticação de usuários
 - HTTPS
 - Validações mais robustas
 - Proteção contra SQL injection (já tem no SQLite, mas reforce)
-

Próximas Melhorias Possíveis

- ☐ Sistema de login/autenticação
 - ☐ Múltiplos usuários
 - ☐ Gráficos de gastos
 - ☐ Relatórios em PDF
 - ☐ Filtros por data/categoria
 - ☐ Metas de economia
 - ☐ Backup automático na nuvem
 - ☐ App mobile (React Native)
 - ☐ Exportar para Excel
-

Suporte

Se tiver problemas:

1. Verifique se todas as dependências foram instaladas (`npm install`)
 2. Confirme que o Node.js está instalado corretamente
 3. Veja os logs do terminal para mensagens de erro
 4. Abra o Console do navegador (F12) para ver erros do frontend
-



Licença

MIT - Use livremente para projetos pessoais ou comerciais!

Desenvolvido com  usando Node.js + SQLite + JavaScript